Interpretation, Valuation, Applications & Semantic Tableaux

Lecture Roadmap

This lecture covers the following topics in Predicate Logic:

Section	Topic	Time	Focus
§6.3.4	Interpretation & Valuation	~20 min	Semantics of PL formulas
§6.3.5	Applications	~10 min	Real-world formalisation
§6.3.5	Semantic Tableaux	~15 min	Proof / refutation method
	Worked Examples & Wrap-up	~5 min	Consolidation

■ *Learning goal: understand how predicate logic formulas get meaning, and how to prove/disprove them systematically.*

Quick Recap: Predicate Logic Syntax

Before semantics, recall the building blocks of a PL formula:

- **Variables**: x, y, z — range over a domain
- **Constants**: a, b, c — specific named individuals
- **Function symbols**: f, g — map domain elements to elements
- **Predicate symbols**: P, Q, R — express properties / relations
- **Connectives**: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$
- **Quantifiers**: $\forall$ (universal), $\exists$ (existential)

Example: $\forall x\ (Human(x) \rightarrow Mortal(x))$

"Every x that is Human is also Mortal" — requires an interpretation to evaluate.

§6.3.4 Interpretation of Predicate Logic

An **interpretation** (also called a *structure* or *model*) I assigns meaning to the non-logical symbols:

Domain D — a non-empty set of individuals (the 'universe of discourse')

Constants — each constant c maps to a specific element $c^I \in D$

Function symbols — each n-ary f maps to a function $f^I : D^n \to D$

Predicate symbols — each n-ary P maps to a relation $P^I \subseteq D^n$

Formal notation

$$I = (D, \{c^I\}, \{f^I\}, \{P^I\})$$

The logical symbols (¬, ∧, ∀, …) have fixed meanings independent of the interpretation.

Interpretation: Worked Example

Let the language have: constant **a**, unary predicate **P**, binary predicate **L**.

Interpretation I■ — domain: natural numbers N

Symbol	Meaning under I■
a	0
P(x)	x is even
L(x, y)	x < y

Evaluating formulas under I■:

- P(a) → 0 is even → **TRUE**
- L(a, a) → 0 < 0 → **FALSE**
- ∃x P(x) → some natural is even → **TRUE**
- ∀x L(a, x) → 0 < every natural → **FALSE** (0 < 0 fails)

■ *The same formula can be true under one interpretation and false under another.*

§6.3.4 Valuation (Variable Assignment)

An interpretation alone is not enough when formulas contain **free variables**. We also need a **valuation** (environment / assignment):

> A valuation σ is a function $\sigma : \text{Var} \to D$ assigning each variable a domain element.
>
> The pair (I, σ) together gives truth values to all formulas, including open ones.

Notation

$$I, \sigma \blacksquare \phi \text{ means: } \phi \text{ is TRUE under interpretation } I \text{ with valuation } \sigma$$

Variable update

$\sigma[x \blacksquare d]$ denotes σ modified so that x maps to $d \in D$ (all other variables unchanged).

$$I, \sigma \blacksquare \forall x\, \phi \text{ iff for all } d \in D\text{: } I, \sigma[x \blacksquare d] \blacksquare \phi$$

$$I, \sigma \blacksquare \exists x\, \phi \text{ iff exists } d \in D\text{: } I, \sigma[x \blacksquare d] \blacksquare \phi$$

Satisfaction Rules (I, σ ■ ϕ)

The satisfaction relation is defined recursively on formula structure:

Formula ϕ	Satisfied iff …
P(t■,…,t■)	$(t■^{I,\sigma}, \ldots, t■^{I,\sigma}) \in P^I$
t■ = t■	$t■^{I,\sigma}$ is the same element as $t■^{I,\sigma}$
¬ψ	I, σ ■ ψ
ψ■ ∧ ψ■	I, σ ■ ψ■ AND I, σ ■ ψ■
ψ■ ∨ ψ■	I, σ ■ ψ■ OR I, σ ■ ψ■
ψ■ → ψ■	I, σ ■ ψ■ OR I, σ ■ ψ■
∀x ψ	for every d ∈ D: I, σ[x■d] ■ ψ
∃x ψ	for some d ∈ D: I, σ[x■d] ■ ψ

Validity, Satisfiability & Entailment

Satisfiable	$\exists$ interpretation I and valuation σ s.t. I,σ ■ ϕ
Valid (Tautology)	$\forall$ I, σ : I,σ ■ ϕ (written ■ ϕ)
Unsatisfiable	No I, σ makes ϕ true
Entailment	Γ ■ ϕ iff every I,σ satisfying all formulas in Γ also satisfies ϕ

Key contrast with propositional logic:

- In prop. logic, validity can be checked with finite truth tables (2^n rows).
- In predicate logic, the domain D can be *infinite* — truth tables don't work.
- This is why we need proof methods like **Semantic Tableaux**.

§6.3.5 Applications of Predicate Logic

Predicate logic is the foundation language for many areas in CS and mathematics:

1. Formal Specification

- Specify software requirements precisely (Z notation, VDM, Alloy).

$$\forall \texttt{req} \in \texttt{Requests}, \exists \texttt{resp} \in \texttt{Responses}: \texttt{handles(system, req, resp)}$$

2. Database Query Languages

- Relational algebra and SQL are rooted in predicate logic.

$$\forall x\ (\texttt{Employee}(x) \land \texttt{Salary}(x) > 50000) \rightarrow \texttt{Senior}(x)$$

3. Artificial Intelligence

- Knowledge representation, Prolog, expert systems.

```
mortal(X) :- human(X). human(socrates).
```

§6.3.5 Applications (cont.)

4. Program Verification (Hoare Logic)

- Pre/post-conditions expressed as predicates.

```
{x > 0} y := x * 2 {y > x}
```

5. Ontologies & Semantic Web

- OWL / Description Logic — reasoning about classes and relationships.

```
∀x (Person(x) → ∃y (hasParent(x,y) ∧ Person(y)))
```

6. Mathematics

- All mathematical theorems are formalised in first-order (or higher-order) predicate logic.

```
∀n ∈ N: n + 0 = n (Peano axioms)
```

■ *Predicate logic is the universal formal language — learning it pays dividends everywhere.*

§6.3.5 Semantic Tableaux — Motivation

We need a **systematic proof procedure** to determine whether a formula is valid, satisfiable, or unsatisfiable.

> **Semantic tableau** (analytic tableau / truth tree) — a tree-based refutation method developed by E.W. Beth (1955) and J. Hintikka (1955).
>
> Tries to show that ϕ is **unsatisfiable** by exhaustively searching for a satisfying model and finding a contradiction on every branch.

Key idea

- To prove $\blacksquare \phi$: assume $\neg\phi$ is satisfiable and derive a contradiction.
- To prove $\Gamma \blacksquare \phi$: assume $\Gamma \cup \{\neg\phi\}$ is satisfiable and derive a contradiction.

Why trees?

- Disjunctions create **branching** — each branch explores one possible 'world'.
- A **closed branch** contains a formula and its negation $\rightarrow$ contradiction.
- A **closed tableau** (all branches closed) $\rightarrow$ proof of validity.

Tableau Rules — Propositional Connectives

Two types of rules determine how branches grow:

α-rules (non-branching — conjunctive)

Formula	Generates (same branch)
T: $\phi \wedge \psi$	T: ϕ and T: ψ
F: $\phi \vee \psi$	F: ϕ and F: ψ
F: $\phi \rightarrow \psi$	T: ϕ and F: ψ
T: $\neg\neg\phi$	T: ϕ
F: $\neg\neg\phi$	F: ϕ

β-rules (branching — disjunctive)

Formula	Left branch	Right branch
T: $\phi \vee \psi$	T: ϕ	T: ψ
F: $\phi \wedge \psi$	F: ϕ	F: ψ
T: $\phi \rightarrow \psi$	F: ϕ	T: ψ

Tableau Rules — Quantifiers

Two additional rule families extend tableaux to **first-order predicate logic**:

γ-rules (universal — can be applied repeatedly)

T: $\forall x\, \phi(x) \rightarrow$ add T: $\phi(t)$ for any term t already in the branch

F: $\exists x\, \phi(x) \rightarrow$ add F: $\phi(t)$ for any term t already in the branch

Must instantiate with **at least one** term; may need multiple instantiations.

δ-rules (existential — introduce a NEW constant)

T: $\exists x\, \phi(x) \rightarrow$ add T: $\phi(c)$ where c is a **fresh constant** not used before

F: $\forall x\, \phi(x) \rightarrow$ add F: $\phi(c)$ where c is a **fresh constant** not used before

The fresh constant is a 'witness' — it represents the unnamed individual that exists.

■ *Apply δ-rules before γ-rules; apply non-branching (α) before branching (β) for efficiency.*

Closure, Termination & Model Extraction

Closing a branch

- A branch is **closed** (marked ×) if it contains both T: ϕ and F: ϕ for some formula ϕ.
- Equivalently: the branch contains a **literal** and its negation.

Tableau outcomes

All branches closed	At least one branch open
→ PROOF of ■ ϕ (or refutation of $\neg\phi$)	→ COUNTER-MODEL (satisfying assignment)

Termination

- For **propositional logic**: always terminates (decision procedure).
- For **predicate logic**: may not terminate (γ-rules can be applied infinitely).
- This reflects the **semi-decidability** of first-order logic (Gödel's completeness theorem).

Extracting a model from an open branch

- Take the domain D = {constants appearing in the branch}.
- For each predicate P: P^I = {tuples (a,…) | T: P(a,…) is on the branch}.

Worked Example 1 — Propositional Tableau

Prove: ■ (p → q) → (¬q → ¬p) (Contrapositive Law)

We assume the negation is satisfiable, i.e., add to the tableau:

F: (p → q) → (¬q → ¬p)

Apply α-rule for F: A → B (A = p→q, B = ¬q→¬p):

T: p → q F: ¬q → ¬p

Apply α-rule for F: ¬q → ¬p:

T: ¬q F: ¬p → T: p

T: ¬q gives F: q. Now apply β-rule for T: p → q:

Left branch:	Right branch:
F: p	T: q
CLOSED (T:p and F:p) ×	CLOSED (T:q and F:q) ×

All branches closed → the formula is **VALID**. ✓

Worked Example 2 — First-Order Tableau

Prove: ∀x P(x) ■ ∃x P(x)

Set up: T: ∀x P(x), F: ∃x P(x)

Step 1 — δ-rule on F: ∃x P(x) (introduce fresh constant a):

```
F: P(a)
```

Step 2 — γ-rule on T: ∀x P(x) (instantiate with a):

```
T: P(a)
```

Step 3 — Branch closure:

```
T: P(a) and F: P(a) → CLOSED ×
```

> Tableau is closed → the sequent is **VALID**. ✓ (∀x P(x) entails ∃x P(x))

Worked Example 3 — Finding a Counter-Model

Is ∃x P(x) ■ ∀x P(x)? Setup: T: ∃x P(x), F: ∀x P(x)

```
δ on T: ∃x P(x) → T: P(a)
```

```
δ on F: ∀x P(x) → F: P(b) (b fresh, distinct from a)
```

No contradiction — branch remains open.

> Counter-model: D = {a, b}, P^I = {a} (P true of a but not b). Sequent is **INVALID**.

Soundness & Completeness of Semantic Tableaux

Soundness

If the tableau for $\Gamma \cup \{\neg\phi\}$ closes, then $\Gamma \blacksquare \phi$.

Intuition: a closed branch represents an impossible world — the contradiction is genuine.

Completeness (Gödel, 1930)

If $\Gamma \blacksquare \phi$, then the systematic tableau procedure will eventually close.

For propositional logic: always terminates.

For predicate logic: terminates if valid; may run forever if invalid (semi-decidable).

Practical heuristics for efficiency

- Apply α**-rules** (non-branching) before β**-rules**.
- Apply δ**-rules** before γ**-rules**.
- Close branches as soon as a contradiction appears.
- Re-apply γ-rules with new constants when no closure found yet.

Lecture Summary

§6.3.4 — Interpretation & Valuation

- An **interpretation** I = (D, …) assigns domain and meanings to non-logical symbols.
- A **valuation** σ : Var $\rightarrow$ D assigns values to free variables.
- Together, (I, σ) defines truth for every PL formula via recursive satisfaction rules.
- Key semantic notions: satisfiable, valid (tautology), unsatisfiable, entailment.

§6.3.5 — Applications & Semantic Tableaux

- Predicate logic underpins specification, databases, AI, program verification, and mathematics.
- Semantic tableaux provide a **tree-based, goal-directed proof method**.
- Rules: α (conjunctive), β (branching/disjunctive), γ (universal), δ (existential/fresh const.).
- Closed tableau $\rightarrow$ **valid**; open branch $\rightarrow$ **counter-model**.
- Sound and complete for PL; decision procedure for propositional logic only.

Next lecture: Proof theory for predicate logic — Natural Deduction and Resolution.

Review Questions

- 1. Define an interpretation for the language {a, f, P} where the domain is Z (integers). Is P(f(a)) true or false in your interpretation?
- 2. State the satisfaction condition for $\forall x\ \psi$ and for $\exists x\ \psi$.
- 3. What is the difference between a formula being *satisfiable* and being *valid*?
- 4. Construct a semantic tableau to show: $\neg\forall x\ P(x) \equiv \exists x\ \neg P(x)$.
- 5. Explain why semantic tableaux for predicate logic may not terminate. What theorem guarantees completeness?
- 6. Give two real-world application domains where predicate logic is directly employed and explain the connection.

Supplementary reading: Britannica 'Formal Logic — Semantic Tableaux'.

Quick Reference Card

Tableau Rule Cheat-Sheet

Rule	Type	Applied to	Produces
α	Non-branch	$T:\phi \wedge \psi$	$T:\phi$, $T:\psi$
α	Non-branch	$F:\phi \vee \psi$	$F:\phi$, $F:\psi$
α	Non-branch	$F:\phi \rightarrow \psi$	$T:\phi$, $F:\psi$
β	Branch	$T:\phi \vee \psi$	Left: $T:\phi$ | Right: $T:\psi$
β	Branch	$T:\phi \rightarrow \psi$	Left: $F:\phi$ | Right: $T:\psi$
γ	Universal (re-usable)	$T:\forall x \phi$	$T:\phi(t)$ for term t
γ	Universal (re-usable)	$F:\exists x \phi$	$F:\phi(t)$ for term t
δ	Existential (fresh c)	$T:\exists x \phi$	$T:\phi(c)$, c fresh
δ	Existential (fresh c)	$F:\forall x \phi$	$F:\phi(c)$, c fresh

Closure: branch closed (×) when $T:\phi$ and $F:\phi$ appear for the same ϕ. Tableau proves validity when ALL branches close.